

GUÍA RESUMEN — Mantenimiento de Dependencias

Resumen integral del mantenimiento de dependencias en un monorepo Flutter + Supabase sin volver a leer el módulo completo. Hilo conductor: Estrategia de versionado → Flujo de actualización → Automatización (Dependabot/Renovate) → SDK y ecosistema → Seguridad y auditoría → Deprecación y EOL → Operación continua (Makefile/CI).

0 Estrategia de versionado

CARET · PINNING · LOCKFILE

Constraint	Rango real	Uso recomendado
<code>^9.1.1</code> (caret)	<code>>=9.1.1 <10.0.0</code>	Librerías Flutter/Dart: <code>flutter_bloc: ^9.1.1</code> , <code>equatable: ^2.1.0</code>
<code>16.3.1</code> (exacto)	Solo esa versión	Frameworks críticos web: <code>next: 16.3.1</code> , <code>react: 19.2.8</code>
SHA (1 commit)	Inmutable	GitHub Actions: <code>actions/checkout@3d3c42e...</code>
<code>any</code> / rangos	Muy abiertos	Solo <code>path</code> y <code>git</code> locales; no en producción

Lockfile

- **Apps:** `pubspec.lock` se **comitea** → todos usan versiones exactas.
- **Librerías:** no se comitea; cada consumidor resuelve.
- **Conflicto de merge:** correr `dart pub get` reconstruye el lockfile.

Overrides (temporales)

`dependency_overrides:` en `pubspec.yaml` fuerza una versión cuando dos paquetes chocan. Desde Dart 3.8 existe `pubspec_overrides.yaml` (no se comitea, va en `.gitignore`) para overrides de desarrollo. Ambos son parches: el objetivo es que el mantenedor upstream actualice su constraint.

Regla de oro: caret para librerías · pinning exacto para frameworks críticos · SHA para Actions · y `dart pub deps` como herramienta de diagnóstico de conflictos.

1 Flujo de actualización

OUTDATED · UPGRADE · FIX · TEST

Columna <code>pub outdated</code>	Significado
Current	Lo que tienes instalado
Upgradable	Máximo con <code>dart pub upgrade</code> (patch/minor)
Resolvable	Máximo que pub resuelve sin tocar <code>pubspec.yaml</code>
Latest	Última publicada (puede ser mayor)

Proceso seguro

1. Leer CHANGELOG + migration guide

2. `dart pub upgrade <paquete>`

3. `dart pub outdated` (verificar)

4. `dart fix --apply` (migraciones del SDK)

5. `flutter analyze`

6. `flutter test -j 1`

7. `build_runner: dart run build_runner build --delete-conflicting-outputs`

8. Commit separado semántico: `fix(deps): update supabase_flutter to 2.17.2`

`dart pub upgrade --major-versions` modifica `pubspec.yaml`. ⚠ **Un paquete a la vez**, nunca todos juntos. Equivalente npm: `npm outdated` → `npm update` → `npm install next@latest` → `npm audit`.

Revisar por paquete: `supabase_flutter` (inicialización), `flutter_bloc` (BlocObserver), `fpdart` (Either/TaskEither), `go_router` (ShellRoute/redirect), `dio` (interceptors/multipart), `isar_community` (regenerar `.g.dart`).

Dependabot — YAML simple, ideal para empezar. Ecosistemas del monorepo: `github-actions` (/), `npm` (`apps/web`), `pub` (`apps/mobile`), schedule `weekly`, límite 5–10 PRs.

Grupos (desde 2023): agrupa familias (`bloc` , `flutter_bloc` , `bloc_test`) o patch updates en un solo PR. Nunca agrupes majors.

Renovate — más potente: `extends: ["config:recommended"]` , `groupName` , schedules por paquete, dashboard, onboarding. Útil cuando necesitas control fino.

Auto-merge (recomendación oficial GitHub)

```
dependabot/fetch-metadata@v3 -> outputs.update-type
gh pr merge --auto --squash "$PR_URL" (solo patch/minor que pasen CI)
```

Auditoría semanal: workflow con `cron` que corre `dart pub outdated --no-dev` , detecta atrasadas y crea un issue automático con `actions/github-script@v9` . Revisión manual SIEMPRE para majors.

Herramienta	Dónde se declara	Cadencia
Flutter SDK	<code>.fvm/flutter_sdk_version</code> (3.47.0) + GitHub Variable <code>FLUTTER_VERSION</code>	Cada release estable (2–3 meses)
Dart SDK	Atado a Flutter (3.47 → Dart 3.13), <code>sdk: ^3.13.0</code>	Automática
Supabase CLI	Sistema / npm global	Mensual (v2.114+)
PostgreSQL	<code>supabase/config.toml</code> → <code>major_version = "17"</code>	Si las features lo requieren
Deno	<code>supabase/config.toml</code> → <code>deno_version = "2"</code>	Cada 3–6 meses
Node.js	<code>.nvmrc</code> (24) + <code>engines</code> (<code>>=22</code>)	Cada ciclo LTS

Estado Node LTS (2026)

Versión	Estado	Acción
Node 18	EOL (2025)	Migrar ya
Node 20	EOL (abril 2026)	Migrar a 22 o 24
Node 22	Maintenance LTS	Mantener / planificar
Node 24 (Krypton)	Active LTS	Usar (recomendado)
Node 26	Current	Solo experimentación

Flutter: usa `fvm install` / `fvm use` , centraliza la versión en GitHub Variables y `flutter-action@v3` lee `{{ vars.FLUTTER_VERSION }}` . Espera 1–2 semanas antes de adoptar un stable nuevo (buena práctica empírica).

- **Árbol:** `dart pub deps --json` + `jq` para extraer dependencias directas.
- **CVE conocidas:** OSV.dev API (`POST /v1/query` , ecosistema `Pub`) y GitHub Advisory Database.
- **Web:** `npm audit` / `npm audit --audit-level=high` / `--json` para CI.
- **Dependabot security updates:** PRs automáticos por CVE (Settings → Code security).

Matriz de respuesta CVSS

Severidad	Acción	Plazo
Critical (9.0–10.0)	Actualizar ya · parche manual si no hay fix	< 24h
High (7.0–8.9)	Próximo ciclo de trabajo	< 1 semana
Medium (4.0–6.9)	Próximo sprint	< 1 mes
Low (0.1–3.9)	Documentar · revisión trimestral	< 3 meses

Gate en CI: por cada paquete del árbol (`nombre@versión`) consulta OSV.dev; si hay `vulns` , falla el build. Es lo que convierte la auditoría en algo reactivo, no manual.

Señales: sin commits >12 meses, issues sin respuesta, paquete que no soporta la última Dart, dependencias obsoletas en cadena (heurística, no regla formal).
Ejemplos reales: `hive` → `isar_community`, `pennylane` → `fpdart`, `built_value` → `freezed` / `json_serializable`, `dartson` → `json_serializable`.

Proceso de migración

Investigar alternativa → rama separada → capa de compatibilidad (wrapper)
→ migración gradual (feature flags, un módulo a la vez)
→ dart pub remove <paquete> → tests completos → ADR + guías del equipo

EOL de Flutter/Dart

Solo la última stable (3.47.0) tiene soporte activo garantizado; 3.44.0 es la anterior. Sin parches de seguridad garantizados para versiones viejas. Cadencia ~3 meses entre releases.

Tras actualizar la constraint del SDK (`sdk: ^3.13.0`), corre `dart fix --apply`. Documenta cada reemplazo como **ADR** (contexto, decisión, consecuencias).
Objetivo del monorepo: nivel **Gold** (Flutter stable + paquetes en minor más reciente).

Target / workflow	Qué hace
make audit	Auditoría completa: mobile (<code>pub deps --json</code>) + web (<code>npm audit</code>) + Actions sin pin
make dep-report	Reporte markdown del estado de dependencias
make check-constraints	Detecta conflictos y <code>dependency_overrides</code> no temporales
make dep-tree / deps-security	Árbol completo / escaneo OSV por paquete
CI semanal	<code>cron</code> + <code>flutter-action@v3</code> (3.47.0) + <code>setup-node@v7</code> (24); crea issue si hay atrasadas
Gate en PRs	Bloquea si hay high/critical en <code>npm audit</code> o <code>dependency_overrides</code> persistentes

Checklist

- ✓ **Por PR:** analyze sin errores, CI verde.
- ✓ **Semanal:** `make audit`, mergear patch/minor que pasen CI.
- ✓ **Mensual:** `make dep-report`, issues de seguridad, lockfiles al día.
- ✓ **Trimestral:** paquetes deprecados, nueva stable de Flutter, docs.
- ✓ **Anual:** auditoría completa, EOL de Node, Flutter 2+ releases atrás.

Madurez: estos targets convierten el mantenimiento de rutina en algo automático, medible y con dueño en el equipo.

A

Anexo · Índice del módulo (para profundizar)

Necesitas	Archivo
Caret, pinning exacto, SHA, lockfile, conflictos, overrides	01-fundamentos-dependencias.md
pub outdated/upgrade, --major-versions, dart fix, build_runner, casos por paquete	02-flujo-actualizacion.md
dependabot.yml, grupos, Renovate, auto-merge, auditoría semanal	03-automatizacion-dependabot-renovate.md
FVM, centralización de versiones, Supabase CLI, .nvmrc, engines, LTS Node	04-mantenimiento-sdk-ecosistema.md
dart pub deps --json, OSV, GitHub Advisory DB, npm audit, CVSS, gates	05-seguridad-auditoria.md
Señales de deprecación, migración con wrapper, EOL Flutter/Dart, ADR	06-deprecacion-eol-migraciones.md
Makefile targets, CI de monitoreo, issue automático, checklist regular	07-makefile-ci-objetivos.md

Mantenimiento de dependencias en una línea: declara con intención (caret/SHA/pinning) → actualiza con flujo seguro → automatiza con bots + auditoría semanal → mantén el SDK sincronizado → responde por CVSS → migra con ADR ante deprecación → y opera con Makefile + CI. · Regla de oro: cada actualización debe pasar por CI y tener un mensaje semántico.